Chapter 1
Why CodeQL and How It Works

Download this chapter as a PDF for offline reading: Chapter-01-Why-CodeQL.pdf

CodeQL is more than another static analysis tool. It is a semantic code analysis engine that treats source code as queryable data. This chapter explains why that distinction matters, how CodeQL overcomes the limitations of traditional approaches, and what the analysis pipeline looks like from source code to actionable findings.

The Limits of Traditional Static Analysis

Traditional static application security testing (SAST) tools have served the industry for decades. Most of them work by matching syntactic patterns—regular expressions, abstract syntax tree (AST) fragments, or rule-based heuristics—against source code. A classic example is a rule that flags any call to `innerHTML` or `eval` as potentially dangerous.

These approaches have clear advantages: they are fast, easy to understand, and require relatively little infrastructure. However, they also share well-known limitations:

- **High false-positive rates.** Pattern matching cannot distinguish safe uses from unsafe ones. An `innerHTML` assignment that receives a constant string is flagged the same way as one that receives unsanitized user input.
- **Limited interprocedural reasoning.** Most traditional tools struggle to track data across function boundaries, files, or library calls. A vulnerability that spans three functions and two files is often invisible to them.
- **Difficulty expressing complex properties.** Questions such as “does attacker-controlled data reach this sensitive operation without sanitization?” require data-flow analysis. Pattern matching can only approximate the answer.
- **Poor support for variant analysis.** When a researcher discovers one instance of a bug, finding every logical variant across a large codebase is a manual, error-prone process.

These limitations are not merely theoretical. They translate into missed vulnerabilities, alert fatigue for developers, and incomplete remediation. CodeQL was designed to address them by shifting from pattern matching to semantic analysis.

Code as Queryable Data

The central idea behind CodeQL is simple to state and powerful in practice: *treat code as data*. Instead of scanning source files for textual or syntactic patterns, CodeQL extracts a rich relational representation of the codebase and stores it in a database. Queries written in the QL language then run against that database.

A CodeQL database contains more than a simple AST. It includes:

- **Abstract syntax trees** that capture the hierarchical structure of the source code.

- **Control-flow graphs** that represent the possible paths of execution through functions and methods.
- **Data-flow graphs** that track how values propagate through variables, function calls, and operations.
- **Semantic information** such as name binding, type information, and call graphs.

Because the representation is relational, a query can ask precise questions. For example: “Find every path where untrusted input reaches a SQL query without passing through a sanitizer.” The query does not search for a string; it searches for a relationship that exists (or does not exist) in the extracted data. This is the difference between guessing and proving.

QL itself is a declarative, object-oriented query language inspired by Datalog. You describe the properties that a result must satisfy rather than the steps the engine should take to find them. The CodeQL libraries then provide high-level classes and predicates that abstract over the raw database tables, so most queries work with concepts such as `Function`, `Call`, and `DataFlow::Node` rather than low-level relations.

Variant Analysis

Variant analysis is the process of taking a known vulnerability or bug and systematically searching for similar instances across a codebase (or many codebases). Security engineers have practiced it for years, usually with a combination of grep, manual review, and custom scripts. CodeQL makes the process scalable and precise.

The typical workflow looks like this:

1. Identify a seed vulnerability—either by finding it yourself or by examining a published advisory.
2. Write (or refine) a CodeQL query that captures the essential pattern of the bug: the source of untrusted data, the sink where it becomes dangerous, and any relevant barriers or sanitizers.
3. Run the query against one or more CodeQL databases. The results are logical variants of the original issue, not merely textual matches.
4. Iterate on the query as needed—adding additional flow steps, tightening sources or sinks, or introducing flow states for greater precision.

Because the query encodes semantic understanding rather than surface syntax, it can find variants that look different in the source code but share the same underlying flaw. A path-traversal bug that uses `os.path.join` in one place and string concatenation in another can be caught by the same query. This is why CodeQL is particularly effective for security research and for ensuring that a fix is applied comprehensively.

When a query is ready, it can be shared as a query pack and run at scale via multi-repository variant analysis (MRVA) on GitHub, or locally with the CodeQL CLI. The same knowledge that found one bug can then be applied to hundreds or thousands of repositories.

The Analysis Pipeline

CodeQL analysis follows a three-stage pipeline. Understanding each stage helps you know where time is spent, where customization is possible, and how results are produced.

1. Database Creation

The first step extracts a relational representation of the source code and stores it in a CodeQL database. Each database represents a single language at a specific point in time.

For *compiled languages* (C/C++, C#, Go, Java/Kotlin, Swift, Rust), extraction typically monitors the normal build process. Whenever the compiler is invoked, the extractor captures the abstract syntax tree, name binding, type information, and other semantic data. This is why a successful build is usually required before analysis can begin.

For *interpreted languages* (JavaScript/TypeScript, Python, Ruby), the extractor analyzes the source files directly, resolving imports and dependencies as needed.

After extraction, the relational data, a copy of the source files, and a language-specific schema are assembled into the database directory. You create a database with the CodeQL CLI:

```
codeql database create my-database --language=javascript --source-root=.
```

2. Query Execution

Once the database exists, you run one or more QL queries against it. Queries can come from the standard CodeQL query packs (covering hundreds of CWEs), from community packs, or from queries you write yourself.

The QL compiler optimizes and evaluates the queries against the relational database. Because QL is declarative and supports recursion and aggregates, complex analyses—such as interprocedural data-flow and taint tracking—can be expressed concisely and executed efficiently.

To see the pipeline in action with the smallest possible query, open the companion repository example at `examples/chapter01/hello-world.ql`. The entire query is:

```
/**
 * @kind problem
 * @id mustlearn/hello
 * @name Hello World
 * @description Minimal query that returns a constant string.
 */
select "hello world"
```

Run it against any CodeQL database (even an empty one works for this query). The result is a single row containing the string “hello world.” This demonstrates that a query is simply a specification of results evaluated against the database; later chapters will show how the same model expresses far richer analyses. You can find and run this

example immediately after cloning the companion repository: github.com/rod-trent/MustLearnCodeQL.

You can run queries from the command line or from the CodeQL extension for Visual Studio Code. The VS Code experience is especially useful while developing queries, because it provides result highlighting, path visualization, and an AST viewer.

3. Interpreting the Results

Raw query results are tables of values. The final stage turns those tables into meaningful alerts. Query metadata (such as `@kind problem` or `@kind path-problem`) tells the tooling how to present the findings.

A simple problem query highlights a single location and displays a message. A path query shows the full path from source to sink, which is invaluable for understanding and triaging data-flow vulnerabilities. Results can be exported as SARIF for consumption by GitHub code scanning, IDEs, or other security tools.

This pipeline—extract, query, interpret—is the same whether you are analyzing a single repository locally or running multi-repository variant analysis across thousands of open-source projects. The difference is scale and automation, not the underlying model.

Summary

Traditional static analysis is limited by its reliance on patterns and its difficulty with interprocedural reasoning. CodeQL overcomes these limits by extracting a full semantic model of the code—syntax, control flow, and data flow—into a queryable database. Queries written in QL can then express precise security and quality properties, and the same queries power both local analysis and large-scale variant analysis.

In the next chapter we will set up the tools and create your first CodeQL database. From there, the rest of the book builds the skills you need to write, refine, and share your own queries. All runnable examples referenced in the book live in the companion repository under `examples/chapterNN/`.

Companion repository: github.com/rod-trent/MustLearnCodeQL